

CS534 : TP Recherche documentaire

Collection de test.....	2
Impact de la langue.....	3
Préparation des fichiers.....	3
Prise en main du processus en python.....	4
Calcul des valeurs classiques - dictionnaire des termes.....	5
Vocabulaire.....	5
DF - Fréquences documentaire.....	5
Analyse de la collection.....	6
Script count et termes fréquents.....	6
Vecteurs de termes.....	9
Binaire.....	9
TF fréquence des termes.....	10
TF IDF.....	11
Construction de fichier inversé.....	12
Moteur de recherche.....	14
Sources :.....	17

Collection de test

Le répertoire CACM contient 6 fichiers : `cacm.all`, `cite.info`, `common_words`, `qrels.text`, `query.text` et un `README`

La collection contient au total 3204 documents. Tous les documents sont contenus dans `cacm.all`, séparés par la balise `.I` (suivie de l'identifiant du document) qui contient le titre du documents (balise `.T`), les auteurs (balise `.A`), la date de publication (balise `.B`), l'abstract (balise `.W`), des mots-clés (balise `.K`). Le fichier **`query.text`** contient le texte des requêtes. Chaque requête est précédée d'une balise `.I` (suivi de l'identifiant de la requête), le texte de la requête est contenu dans la balise `.W`, la balise `.N` donne l'auteur de la requête (à ignorer) Le fichier **`qrels.text`** contient les identifiants des documents pertinents pour les requêtes. Chaque ligne donne un couple d'identifiants (requête, document pertinent pour la requête). Enfin, le fichier **`common_words`** contient les mots fréquents de la collection, et servira de stop-list. Le fichier **`cite.info`** contient des informations relatives au codage des citations (balise `.X`) dans le fichier `cacm.all`. Ce fichier est ignoré dans la suite.

Une collection de test est un ensemble de documents textuels spécialement construits pour permettre l'évaluation et l'expérimentation de méthodes de RI. Elle fournit une base commune sur laquelle on peut appliquer, comparer ou analyser des algorithmes de nettoyage, d'indexation ou de recherche. Une collection de test est généralement accompagnée d'un format structuré et de fichiers annexes permettant d'identifier clairement les documents qu'elle contient.

Dans ce TP, la collection utilisée est la collection CACM (Communication of the ACM). Elle se présente sous la forme d'un fichier source `cacm.all` qui regroupe l'intégralité des documents. Chaque document y est décrit à l'aide de balises structurées, comme nous l'avons vu dans la question précédente.

À partir de ce fichier, les scripts fournis (par exemple Decode.pl) extraient pour chaque identifiant .I un fichier séparé contenant les parties pertinentes du document, en particulier le texte issu de la balise .W. La collection finale est donc constituée d'un ensemble de fichiers individuels (un par document), accompagnés d'un fichier Collection listant leurs noms. Cette structure rend la collection exploitable pour les différentes étapes du TP : nettoyage, suppression des mots vides, construction du vocabulaire, calcul des fréquences, représentation vectorielle et indexation.

Impact de la langue

La langue utilisée dans cacm.all est l'anglais, et la collection complète est en anglais. Si elle était multilingue on aurait un problème puisque cela fausserait la pertinence de notre analyse concernant la fréquence d'occurrence des différents mots, puisque les occurrences ne sont pas les mêmes d'une langue à l'autre. En anglais les mots les plus fréquents sont : the, be , to [1] alors qu'en français ce sont le, de, un [2]

Préparation des fichiers

Nous avons écrit trois scripts python pour travailler sur la collection.

1. Enlever des caractères spéciaux et enlever les espaces (clean.py)
2. Enlever les mots vides (empty.py)
3. Application du stemming avec l'algorithme de Porter. (stemming.py)

Ces trois scripts nous permettent donc d'obtenir 3 différentes collections.

Nom du fichier :	clean.py	empty.py	stemming.py
Extension :	.clean	.nowords	.stem
Fonction :	Suppression accents, majuscules et espaces	Suppression des mots vides	Application du stemming

Prise en main du processus en python.

DecodeCACMXX.pl :

Ce programme ouvre le fichier cacm.all et crée les fichiers de contenu de type CACM-XX, où XX est le n° du document, en le découpant selon les balise .l

Clean.pl :

Ce programme ouvre tous les fichiers du dossier qui lui est donné, ici Collection, car ce dossier contient tous nos fichier CACM-XX. Ensuite, il va nettoyer les fichiers de la collection en enlevant les accents, les retours à la ligne, les ponctuations, et les espaces créés inutiles et mettre tout le texte en minuscule. Les fichiers créés en sorties prennent l'extension .flt.

Remove.pl :

Ce programme ouvre tous les fichiers du dossier Collection qui ont l'extension .flt, et enlève tous les stop-words des fichiers CACM-XX.flt, en utilisant la stop-list common-words, le résultat du filtrage est mis dans des fichiers CACM-XX.stp

Par la suite dans le TP, nous utiliserons une seconde version de clean et remove, que nous avons codée en python : clean.py et empty.py. Clean.py agit de la même manière que clean.pl, sauf qu'il génère des fichiers dont l'extension est .clean, comme nous l'avons vu précédemment. Il en va de même pour empty.py, qui génère des fichiers en .nowords après avoir enlevé les stop-words.

Ces différents programme nous permettent donc d'obtenir à la fin trois collections distinctes :

1. Une collection sans caractères spéciaux, accents, ni espaces - **Collection 1**
2. Une collection où on a en plus enlevé les mots vides - **Collection 2**
3. Une dernière collection où on a fait du streaming - **Collection 3**

Dans la suite du TP, nous allons appliquer chacun des programmes que nous allons coder à ces trois collections.

Calcul des valeurs classiques - dictionnaire des termes

Vocabulaire

Avec la fonction `vocabulaire.py`, on vient lire tous les documents qui contiennent l'extension demandée (`.clean` / `.nowords` / `.stem`), et dès qu'on rencontre un nouveau mot, on update notre objet python `set()` qui stocke le vocabulaire. Notre fichier de sortie est de la forme `vocabulary.<file_extension>.txt`, et on renvoie le nombre total de mot de ce fichier.

Collection 1 .clean	Collection 2 .nowords	Collection 3 .stem
10 769 mots	10 414 mots	6 952 mots

DF - Fréquences documentaire

Avec la fonction `df.py`, on cherche à savoir dans combien de document apparaît un mot. Pour cela, on vient lire tous les documents d'une collection donnée, et on utilise cette fois-ci un dictionnaire. Les clefs sont les mots et les valeurs sont le nombre de documents dans lesquels ils apparaissent. Dès qu'on rencontre un nouveau mot, on ajoute cette clef dans le dictionnaire, et si l'entrée existe déjà, on fait +1 à la valeur. Si un mot apparaît plusieurs fois dans le même document, on ne le comptabilise qu'une seule fois. Le résultats de cette fonction est écrit dans un fichier de la forme `df.<file_extension>.txt`

On voit que le terme `Cacm` est présent dans tous les documents sauf un, il n'est donc pas significatif, on peut l'ignorer.

Analyse de la collection

Script count et termes fréquents

Count.py

Le programme count.py permet de compter le nombre d'occurrences de chaque mot dans la collection. Le parcours du fichier Collection permet de connaître le nom de chaque document de la collection de test. Le parcours d'un fichier permet de calculer le nombre d'occurrence d'un même mot. Ce programme écrit dans un fichier de sortie les informations sous la forme : Rang du mot, Compte du mot et Mot trié par ordre croissant des rangs.

Nous avons choisi d'ignorer le mot cacm.

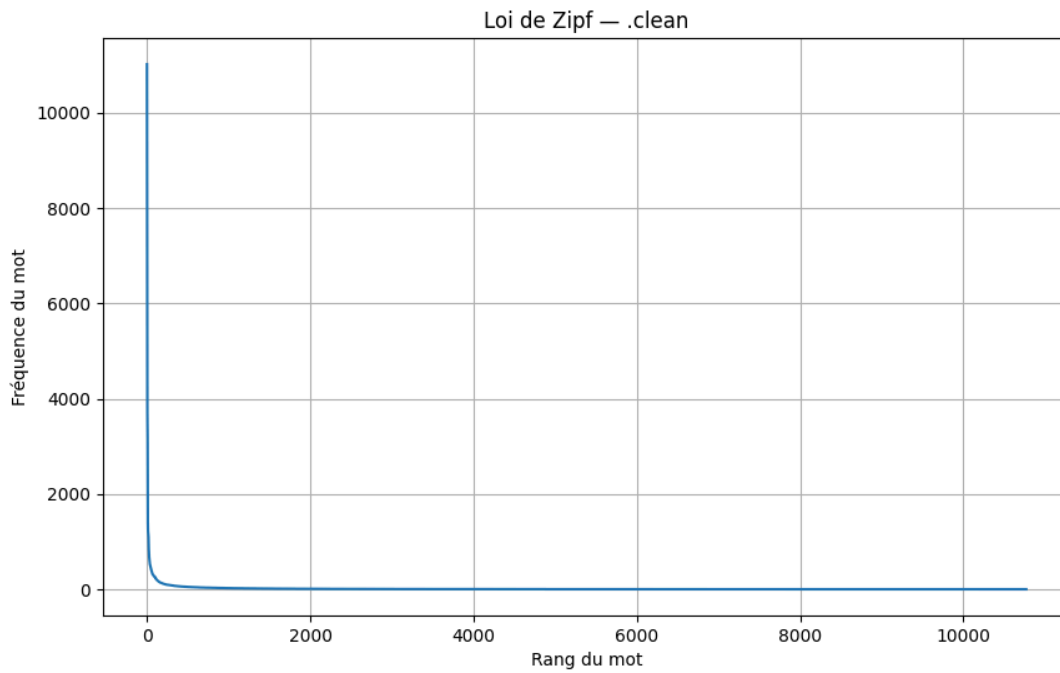
	Collection 1 .clean	Collection 2 .nowords	Collection 3 .stem
Vocabulaire	10 769 mots	10 414 mots	6 952 mots
Mot le + fréquent	the (11 018 fois)	algorithm (1 544 fois)	algorithm (1 867 fois)

TermFreq.py

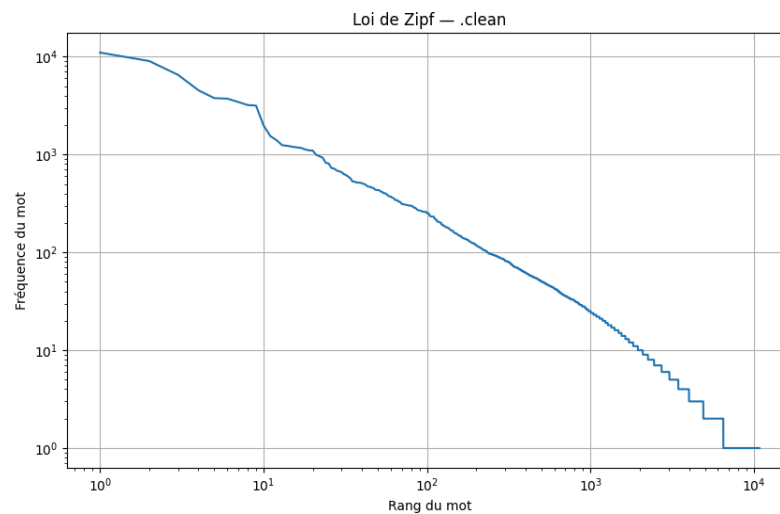
Ce programme permet de calculer le nombre moyen d'apparitions du terme le plus fréquent dans les documents d'une collection : c'est-à-dire trouver la moyenne d'apparition d'un terme quand il apparaît dans un document. Pour cela, on parcourt tous les documents d'une collection donnée, et on compte le nombre de fois où le mot apparaît, puis on divise le nombre d'apparition du mot par le nombre de documents parcourus.

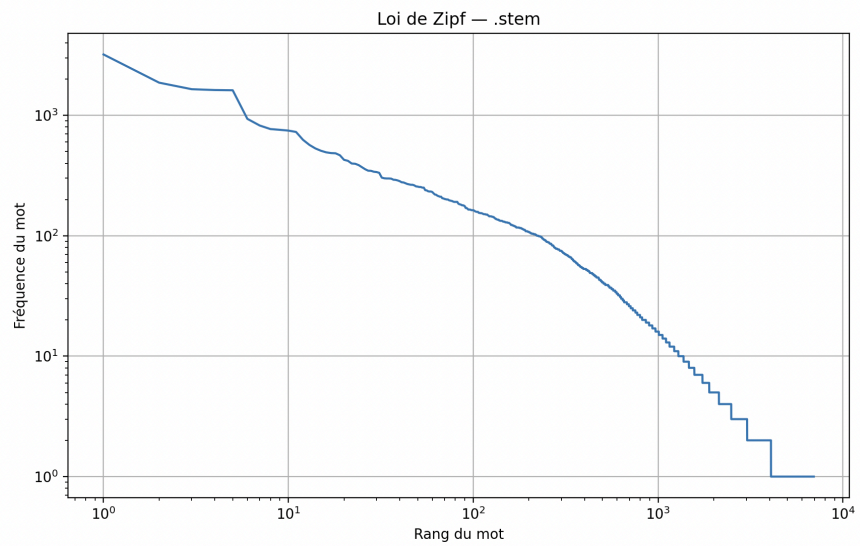
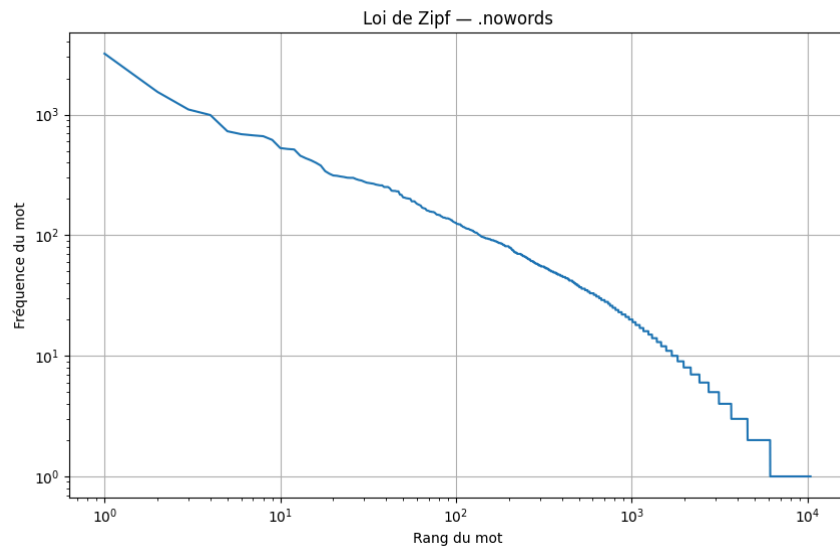
Utilisation des résultats

Nous avons codé le programme plot.py, qui permet de créer un graphique visualisant la fréquence des mots (en ordonnée) en fonction du rang (en abscisse). Ce programme parcourt le fichier count.<file_extension>.txt, et récupère les rang des mots, ainsi que le nombre de fois qu'ils apparaissent.



En plotant une première fois les résultats pour la Collection 1, on s'aperçoit que notre échelle n'est pas bonne, on va donc passer en échelle log-log. En effet, on utilise l'échelle log-log puisque d'après la loi de Zipf, fréquence $\approx 1 / \text{rang}$ donc avec l'affichage log-log on devrait obtenir une droite. Pour rappel, la loi de Zipf est une observation empirique concernant la fréquence des mots dans un texte.





En observant les résultats obtenus pour nos trois collections, on voit que la loi de Zipf est respectée.

Vecteurs de termes

Binaire

L'objectif est de représenter chaque document sous forme d'un vecteur binaire indiquant uniquement la présence ou absence d'un terme.

Pour construire ce vecteur, on utilise la structure suivante :

1. Le vocabulaire est chargé depuis un fichier vocabulaire<extension>.txt, contenant la liste des mots uniques.
2. Chaque mot est associé à un identifiant numérique correspondant à son rang dans le vocabulaire.
3. Pour chaque document, on parcourt le texte et on vérifie si un mot appartient au vocabulaire.
4. Si un terme apparaît au moins une fois, on génère la paire termID:1.

On a un script générique nommé vecteurBinaire.py, capable de fonctionner avec les trois versions des documents (.clean, .nowords, .stem) : l'extension des fichiers est passée en argument, ce qui permet d'utiliser automatiquement le vocabulaire correspondant.

```
CACM-44 1:1 6698:1
CACM-45 3707:1
CACM-46 1:1 385:1 505:1 775:1 1194:1 1228:1 1315:1 1695:1 1827:1 1869:1 2544:1 2552:1 3352:1 3707:1 4607:1 4646:1 4661:1
4989:1 5596:1 5677:1 5988:1 6427:1 6589:1 6663:1 6698:1 6924:1 7210:1 7440:1 7513:1 7516:1 7555:1 7559:1 7562:1 7563:1 7822:1
8172:1 8766:1 9410:1 9568:1 9684:1 9746:1 9748:1 9781:1 9890:1 10577:1
CACM-47 9836:1
CACM-48 1:1 385:1 452:1 517:1 561:1 848:1 860:1 1228:1 1272:1 1631:1 1695:1 1697:1 1730:1 1823:1 2120:1 2295:1 2450:1 2544:1
2545:1 2575:1 2830:1 2881:1 2922:1 3100:1 3236:1 3320:1 3624:1 3654:1 3707:1 3994:1 4242:1 4346:1 4510:1 4516:1 4607:1 4910:1
4989:1 5014:1 5636:1 6326:1 6663:1 6698:1 6702:1 6965:1 7078:1 7093:1 7095:1 7601:1 7690:1 7888:1 7889:1 7965:1 8818:1 8905:1
9032:1 9198:1 9201:1 9665:1 9748:1 9781:1 9815:1 9836:1 10047:1 10265:1 10385:1 10627:1 10636:1 10640:1 10664:1 10753:1
CACM-49 385:1
CACM-50 3707:1 9748:1
CACM-51 6663:1 9748:1
CACM-52 361:1 3707:1 6698:1 9748:1
CACM-53 6663:1 9748:1
CACM-54 1:1 3707:1
CACM-55
CACM-56
CACM-57 385:1
CACM-58 1:1 385:1 630:1 1297:1 1823:1 1973:1 1986:1 2598:1 2601:1 2671:1 2907:1 3318:1 3934:1 5705:1 6663:1 6915:1 7676:1
7733:1 9032:1 9469:1 9748:1 10125:1 10275:1
CACM-59 385:1 561:1 2038:1 4607:1 6663:1 9748:1
CACM-60 1:1 6663:1
CACM-61
CACM-62
CACM-63 20:1 378:1 385:1 450:1 561:1 750:1 955:1 1228:1 1519:1 1844:1 2342:1 2533:1 2552:1 2632:1 3954:1 6128:1 6656:1 6663:1
6915:1 9748:1 9836:1 10014:1 10721:1
CACM-64 385:1 6698:1
CACM-65
CACM-66 1:1 3707:1
CACM-67
CACM-68 1:1 117:1 385:1 505:1 561:1 1605:1 1827:1 2076:1 2331:1 2597:1 2717:1 3597:1 3622:1 4607:1 4805:1 4910:1 4989:1
5417:1 5698:1 5777:1 6663:1 6698:1 6725:1 6809:1 7434:1 7555:1 7563:1 7635:1 7914:1 7993:1 8045:1 9323:1 9417:1 9747:1 9748:1
9836:1 10195:1 10485:1 10554:1 10610:1
CACM-69 1123:1 1386:1 1668:1 1829:1 2548:1 3707:1 3994:1 4257:1 4607:1 4989:1 6577:1 6663:1 7563:1 9203:1 9748:1 10195:1
```

Extrait de vecteurBinaire.clean.txt

TF fréquence des termes

Cette seconde version améliore la représentation en prenant en compte la fréquence des termes dans le document. C'est la même idée que la version binaire, mais au lieu d'indiquer uniquement la présence d'un terme, on compte le nombre de fois qu'il apparaît.

1. On charge le vocabulaire et construit le dictionnaire mot -> ID.
2. On parcourt chaque document et compte le nombre d'apparitions de chaque terme (TF).
3. Pour chaque terme du document, la paire : termID:tf est construite
4. Les id sont triés

Le script correspondant est vecteurTF.py, également générique et fonctionnant pour .clean, .nowords et .stem.

```
CACM-2466
CACM-2467
CACM-2468
CACM-2469 1:2 999:1 1562:2 3829:1 4124:1 4842:1 5821:3
CACM-2470 1:2 236:1 479:6 743:1 1442:4 1570:1 1940:1 2246:2 2321:1 2433:1 3082:1 3522:2 4468:1 4855:1 4873:1 5583:1 5648:2
5684:1 5703:1 6130:4 6282:1 6593:1
CACM-2471 1:4 94:1 602:1 918:1 2340:2 3485:1 4493:1 4674:1 4855:4 6282:1 6340:1 6593:1 6887:1
CACM-2472
CACM-2473
CACM-2474
CACM-2475
CACM-2476
CACM-2477 1:1
CACM-2478
CACM-2479 1:1 67:2 188:1 1166:1 1394:2 2135:1 2247:1 2271:1 6496:1 6868:1
CACM-2480 1:5 188:1 479:3 2101:1 2321:1 4309:1 4458:1 4674:1 4791:1 4823:1 4841:1 4855:2 5583:1 5678:1 5722:1 5981:1 6184:2
6682:1
CACM-2481 1:4 739:4 901:1 1166:1 2110:2 2267:1 4855:2 6130:1 6534:1 6676:1
CACM-2482 1:1 479:1 1173:1 1462:1 4458:1 4803:1 4835:1
CACM-2483 1:9 188:1 479:1 962:7 1032:1 1052:1 1060:1 1123:1 1141:1 1334:1 1579:1 1940:1 2067:1 2340:2 2537:1 2646:1 2788:2
3438:1 3568:1 3623:1 3985:1 4608:1 4675:1 4855:1 5028:1 5519:1 6294:1 6386:5 6887:1
CACM-2484 1:12 145:2 188:3 456:2 479:2 1442:2 1562:1 1851:1 2246:1 2421:2 2488:1 3829:1 4835:1 4841:1 5583:3 6386:3 6593:1
6669:1
CACM-2485 1:2 930:1 1166:3 1442:1 4567:1 5871:2 6130:1
```

Extrait de vecteurTF.stem.txt

On voit dans cet exemple que pour certains documents, il n'y a aucun terme présent dans le vocabulaire, et on voit que pour le document 2484, un terme est présent 12 fois, c'est le maximum pour cette collection.

TF IDF

Cette troisième version combine la fréquence locale du terme dans le document (TF) et son importance globale dans la collection via la fréquence documentaire DF, provenant du fichier `df<extension>.txt`.

L'objectif est de donner plus de poids aux termes informatifs et moins aux termes trop fréquents. On calcule l'IDF selon la formule

$$IDF(t) = \log(N / DF(t))$$

où N est le nombre total de documents et t un terme.

Enfin on calcule le TF-IDF selon la formule

$$TF-IDF(t, d) = TF(t, d) * IDF(t)$$

Le script associé est `vecteurTFIDF.py`, et est paramétrable par extension (`.clean/.stem/.nowords`)

```
CACM-2160 1:22.137024 142:4.011712 479:24.216466 2164:26.743444 2421:6.973543 6751:4.776318
CACM-2161 1:7.379008
CACM-2162 1:7.379008 183:4.180335 188:7.379008 569:3.977811 3504:3.189353 4268:2.202858 4842:3.715446
CACM-2163 1:22.137024 243:3.174316 1392:6.280396 2271:4.981113 2486:5.874931 4102:3.251874 4458:1.980845 5307:5.027633 6086:4.
488636 6593:5.433098
CACM-2164 1:22.137024 145:1.784171 1021:6.855529 3406:17.978141 4835:1.965132 4873:4.180335 5032:7.379008 5809:3.539556
CACM-2165 1:14.758016 142:4.011712 1728:5.127716 2158:7.989236 3716:5.674260 4458:1.980845
CACM-2166 1:14.758016 145:0.892085 188:7.379008 1214:3.897768 3623:3.204621 4447:6.685861 4458:1.980845 4835:3.930265 5211:3.
977811 6593:10.866196
CACM-2167 1:36.895041 130:7.379008 188:14.758016 479:8.072155 1159:7.379008 1225:3.189353 1562:2.492425 1657:2.197225 2192:8.
072155 3642:3.380807 4689:6.031819 5521:8.072155 5818:6.973543 6451:3.144902 6596:3.054875 6669:8.668971
CACM-2168 1:14.758016 3504:6.378707 4458:1.980845 5358:3.677706 5716:3.427764 6130:1.557443 6593:10.866196
CACM-2169 1:29.516033 183:4.180335 479:8.072155 1442:2.166793 1816:4.516807 3882:4.120912 4458:1.980845 5032:7.379008 5358:3.
677706 5825:6.462717 6130:3.114885 6294:2.051132
CACM-2170 4458:1.980845 4953:5.874931
CACM-2171
```

Extrait de vecteurTFIDF.stem.txt

Construction de fichier inversé

Un index inversé associe chaque terme à la liste des documents dans lesquels il apparaît. Pour le construire, on va suivre l'algorithme proposé dans le sujet du TP à savoir :

1. Extraction des paires d'identifiants {Terme, Document}, avec une passe complète sur l'ensemble des documents de la collection
2. Tri des paires suivant les idTerme, et les idDocuments.
3. Regroupement des paires pour établir, pour chaque terme, la liste des docs.

Nous avons codé cette méthode dans un script générique nommé `indexInverse.py`, qui s'adapte automatiquement à la version de la collection passée en argument (`.clean`, `.nowords`, `.stem`), et récupère les documents concernés via leur extension, et le fichier `Collection`.

Le script génère un fichier `indexInverse<extension>.txt` contenant l'index inversé complet sous la forme : `IdTerme : IdDocument1 IdDocument2 IdDocument3 ...`

```
● (venvTP) → TPRIetu2020 python3 indexInverse.py .nowords
Vocabulaire : 10414 termes
Lecture de la liste des documents...
Fichier : ./cacm/Collection
Nombre de documents à traiter : 3204
Documents à traiter : 3204 fichiers
Extraction des paires : 47425 paires
Index inversé généré -> indexInverse.nowords.txt
```

Sortie de `indexInverse.py`, génération du fichier index Inverse pour la collection sans les mots vides.

	Collection 1 .clean	Collection 2 .nowords	Collection 3 .stem
Vocabulaire	10 769 mots	10 414 mots	6 952 mots
Nombre de paires	82 517	47 425	18 673

```
4 : 1878
5 : 2534
7 : 1062
12 : 3203
17 : 605 644 1010 1033 1155 1455 1502 1541 1565 1571 1698 1856 1938 1960 2138 2629 2644 2820 2850 2876
3066 3080 3145 3150
18 : 675 1170 1489 1805 1878 2106 2195 2297 2319 2378 2692 2719 2795 2862 3087 3139
23 : 1784
24 : 1487 1751 1805 1862 2665 2695 3140
25 : 1529 1553 1874 2245 2625 2807 3057
26 : 1216 3171
27 : 2341
29 : 1469 1678 1747 2684 2705 2851 2931 2940 2950 2958 3000 3031 3103 3105 3128 3133
30 : 1087
31 : 1846 2184 2265 2738 2939 2940 2957 3031 3100 3103
32 : 1749 2470 2709 2931 2939 2957 3100
33 : 2470
34 : 1349 1591 1771 1862 2451 3022 3146
35 : 2181
36 : 1873
```

Contenu de *indexInverse.nowords.txt*

Moteur de recherche

Dans cette partie, on va récupérer la requête de l'utilisateur qui souhaite interroger la collection, et la parser pour récupérer les mots qu'il cherche. Le moteur de recherche fonctionne en trois grosses étapes :

1. On convertit la requête en vecteur, il est alors nécessaire de tokeniser la requête selon le même modèle que pour la collection demandée.
2. On interroge l'index inversé pour chaque terme de la requête, on récupère les documents dans lesquels les termes sont présents.
3. On utilise une fonction de scoring pour déterminer quels sont les documents les plus pertinents, cela peut être fait avec le TF ou le TF-IDF.

On commence par faire un premier script, *queryVector.py* qui transforme les mots de la requête

```
(venvTP) → TPRIÉtu2020 python3 queryVector.py .clean "information retrieval system research"

Lecture de la liste des documents...
Fichier : ./cacm/Collection
Nombre de documents à traiter : 3204
Documents à traiter : 3204 fichiers
Nombre total de documents : 3204
{4721: 2.6254179365219397, 8260: 3.7414219679019185, 9568: 1.8456186389007843, 8184: 3.99461786428253
03}
```

Sortie de queryVector.py pour la recherches "information retrieval system research" dans la collection .stem

Dans la requête précédente, on voit le termeID de chaque mots de la requête ainsi que son poids calculé avec TF-IDF

La vectorisation de la requête permet de la transformer dans le même espace vectoriel que les documents (même vocabulaire, mêmes identifiants de termes, mêmes pondérations TF ou TF-IDF). Cette étape est indispensable pour pouvoir calculer un score de similarité entre la requête et chaque document. Sans cette représentation vectorielle, il serait impossible d'utiliser les fonctions de scoring, qui reposent sur un produit entre les poids des termes de la requête et ceux des documents.

Dans un second temps, on vient réaliser un script *searchTF.py* qui intègre cette logique de transformation de la requête en vecteur avant d'interroger l'index inversé. Dans ce script, la fonction de scoring se fait avec TF selon : $\text{score}(\text{doc}) = \text{somme des TF}(t, \text{doc})$ où t est un terme de la recherche.

Pour chaque terme de la requête, on récupère les documents correspondants dans l'index inversé, puis on accumule les fréquences des termes dans chaque document.

Les documents sont ensuite triés par score décroissant.

```

● (venvTP) → TPRIEt2020 python3 searchTF.py .nowords "information retrieval system research" 5

Index inversé chargé : 6312 termes
TF des documents chargé : 3204 documents
Doc 2288 – score 10.0000
Doc 3134 – score 10.0000
Doc 1194 – score 9.0000
Doc 1591 – score 9.0000
Doc 1412 – score 8.0000
Résultats écrits dans resultatsTFquery.html

```

Sortie de searchTF.py pour la recherche “information retrieval system research” dans la collection .nowords

Ici, on affiche les N meilleurs résultats, ici on a choisit 5 mais cette valeur est à la décision de l'utilisateur. Les 5 meilleurs documents sont présentés, et le résultat est écrit au format HTML dans le documents resultatsTFquery.html, généré automatiquement, avec des liens cliquables vers les documents.

Dans un troisième temps, on vient réaliser un script searchTFIDF.py qui reprend le fonctionnement du script précédent, mais cette fois ci la fonction de scoring se fait avec TF-IDF selon : $\text{score}(\text{doc}) = \text{somme des } \text{TF}(t, \text{doc}) \times \text{IDF}(t)$, où t est un terme de la recherche. Pour chaque terme de la requête, on récupère les documents correspondants dans l'index inversé, puis on accumule les fréquences des termes dans chaque document. Les documents sont ensuite triés par score décroissant.

```

● (venvTP) → TPRIEt2020 python3 searchTFIDF.py .clean "chemical research" 5
Lecture de la liste des documents...
Fichier : ./cacm/Collection
Nombre total de documents : 3204
Doc 1003 – score 83.5334
Doc 2402 – score 41.7667
Doc 2721 – score 41.7667
Doc 3135 – score 41.7667
Doc 3194 – score 41.7667
Résultats enregistrés dans resultatsTF-IDFquery.html

```

Sortie de searchTFIDF.py pour la recherche “chemical research” dans la collection .clean

```

1  <h1>Résultats de la recherche</h1>
2  <ul>
3  <li><a href="./Collection/1003.clean">Document 1003</a> – score 83.5334</li>
4  <li><a href="./Collection/2402.clean">Document 2402</a> – score 41.7667</li>
5  <li><a href="./Collection/2721.clean">Document 2721</a> – score 41.7667</li>
6  <li><a href="./Collection/3135.clean">Document 3135</a> – score 41.7667</li>
7  <li><a href="./Collection/3194.clean">Document 3194</a> – score 41.7667</li>
8  </ul>

```

Exemple d'un fichier HTML généré pour la recherche précédente

Sources :

- [1] https://en.wikipedia.org/wiki/Most_common_words_in_English
- [2] <https://eduscol.education.fr/document/15655/download>